

DEV-588:Research Redis security best practices

(authentication, ACLs, network binding)

Department: DevOps

Prepared by: Thanusha Bai V

Date: 17-sept-2026

Task: DEV-587-Redis Security Hardening

1. Executive Summary

This report documents the research findings on Redis security best practices for Glynac's deployments. Redis was originally designed for trusted, internal network environments, but production deployments require explicit security hardening. A misconfigured Redis instance exposed to the internet can result in remote code execution and complete server compromise through exploits like the CONFIG SET file-write technique. This research covers authentication, Access Control Lists (ACLs), network binding, and supplementary hardening measures that form the foundation for Glynac's Redis security standard.

2. Authentication & Access Control

2.1 Legacy Authentication (requirepass)

The requirepass directive sets a global password for Redis. All clients must authenticate before issuing any commands. However, Redis can test up to 1 million passwords per second, making short passwords vulnerable to brute-force attacks. Passwords should be at least 32 characters long and generated from cryptographically secure sources.

2.2 ACL-Based Authentication (Redis 6+)

Redis 6 introduced Access Control Lists (ACLs), providing fine-grained access control over commands, keys, and channels. ACLs are the recommended approach for new deployments, as they

allow multiple users with distinct permissions rather than a single global password.

Core ACL capabilities include:

- **User management:** Create, modify, and delete named users. Users can be enabled (on) or disabled (off).
- **Password rules:** Add passwords with >password or remove with <password. Multiple passwords per user are supported.
- **Key pattern restrictions:** Use ~pattern to grant access to keys matching a glob pattern. Redis 7+ supports read-only (%R~) and write-only (%W~) patterns.
- **Command permissions:** Allow or deny command categories (e.g., +@read, -@dangerous) or individual commands (e.g., +GET, -CONFIG).
- **Channel restrictions:** Control Pub/Sub access with &pattern rules.

Example ACL configuration:

```
user default off
user app on >StrongPassword32Chars ~app:* +@read +@write -@dangerous
user readonly on >StrongPassword32Chars ~* +@read
user admin on >StrongPassword32Chars ~* +@all
```

The default user should always be disabled in production to prevent unauthenticated access.

2.3 Password Rotation

Redis supports multiple passwords per user, enabling zero-downtime rotation. The process is: add new password → update application configs → restart services → remove old password → run ACL SAVE.

3. Network Binding & Exposure Control

3.1 The bind Directive

Redis should never bind to 0.0.0.0 unless protected by a firewall that restricts access. The bind directive specifies which network interfaces Redis listens on.

Secure configurations:

```
# Localhost only (development)
bind 127.0.0.1

# Specific private interfaces (production)
bind 127.0.0.1 10.0.1.5
```

3.2 Protected Mode

Protected mode is a safety layer that refuses connections from non-loopback addresses when no password is set. It is enabled by default (protected-mode yes) and should remain enabled unless Redis is protected by explicit firewall rules and authentication.

3.3 Firewall Rules

Restrict Redis port access to trusted hosts only:

```
iptables -A INPUT -p tcp --dport 6379 -s 10.0.0.0/24 -j ACCEPT
iptables -A INPUT -p tcp --dport 6379 -j DROP
```

3.4 Unix Domain Sockets

For local-only connections, Unix domain sockets provide better security than TCP:

```
unixsocket /run/redis.sock  
unixsocketperm 700
```

4. TLS Encryption

For any cross-network Redis traffic, TLS encryption should be enabled to protect data in transit.

Configuration:

```
tls-port 6380  
port 0  
  
tls-cert-file /etc/redis/redis.crt  
tls-key-file /etc/redis/redis.key  
tls-ca-cert-file /etc/redis/ca.crt  
tls-auth-clients yes
```

Setting port 0 disables the non-TLS listener entirely. Client certificate authentication (tls-auth-clients yes) enforces mutual TLS (mTLS) in production.

5. Dangerous Command Restrictions

5.1 Commands to Disable or Rename

The following commands pose significant security risks and should be disabled or restricted:

Command	Risk
CONFIG	Enables file-write RCE via CONFIG SET dir + dbfilename
FLUSHALL / FLUSHDB	Mass data deletion
DEBUG	Information disclosure, crash triggers
KEYS	Performance impact, data enumeration
EVAL / EVALSHA	Lua script injection vectors
SLAVEOF / REPLICAOF	Replication hijacking
SHUTDOWN	Denial of service

5.2 ACL-Based Command Restrictions

In Redis 7+, ACLs are the preferred method for command restriction, as `rename-command` is deprecated:

```
user app on >password ~* +@read +@write -@dangerous -CONFIG -DEBUG -  
FLUSHALL
```

6. OS-Level & Deployment Hardening

6.1 Run as Non-Root User

Redis should never run as root. A dedicated system account should be created:

```
useradd -r -s /sbin/nologin redis
chown -R redis:redis /var/lib/redis /var/log/redis
```

6.2 Configuration File Permissions

The Redis configuration file contains passwords and should be restricted:

```
chmod 640 /etc/redis/redis.conf
chown redis:redis /etc/redis/redis.conf
```

6.3 Container Hardening

Default Redis containers are insecure: redis:7-alpine scores 48/100 (grade D) by default, running as root with full capabilities and a writable filesystem. Hardening requires:

```
docker run -d --name redis-hardened \
  --user 65532:65532 \
  --cap-drop=ALL \
  --read-only --tmpfs /tmp \
  --network=none \
  -v redis-data:/data \
  redis:7-alpine
```

This combination eliminates the file-write RCE primitive and prevents container escape from landing as host root.

7. Summary of Key Findings

Area	Best Practice	Priority
Authentication	Use ACLs, disable default user, 32+ char passwords	Critical
Network	Bind to private interfaces, enable protected-mode, firewall rules	Critical
TLS	Enable TLS with client certificate authentication	High
Commands	Disable/restrict CONFIG, FLUSHALL, DEBUG, EVAL via ACLs	Critical
OS	Non-root user, restricted config permissions	High
Containers	Non-root UID, dropped capabilities, read-only rootfs	High

8. Recommendations for Glynac

1. **Adopt ACL-based authentication** as the standard. Create service-specific users with minimal permissions (to be detailed in DEV-590).
2. **Bind Redis to private interfaces only** and enable protected mode.
3. **Enable TLS** for all cross-network communication.

4. **Disable or restrict dangerous commands** using ACL rules.
 5. **Run Redis as a non-root user** with restricted file permissions.
 6. **Apply container hardening flags** if using Docker or Kubernetes deployments.
-

This research forms the foundation for subsequent subtasks in the Redis Security Hardening project.

END OF REPORT